Assessment 5 — Full Project Documentation

Project title: Metabolic Risk Digital Twin (NHANES 2017–2018)

This document is the detailed "single source of truth" for the project as it exists now: data → training → artifacts → API → React dashboard → testing/validation. It is written for assessment submission and demo-readiness.

Important constraint: existing research notebooks in `notebooks/` were not modified (they back your paper).

1) Executive summary

This project builds a Patient Digital Twin for metabolic risk screening using NHANES 2017–2018. A patient is represented as a compact, clinically meaningful feature vector. The system provides:

- A reproducible ML pipeline (train, tune, export)
- A FastAPI service for inference and analysis
- A React dashboard UI that supports prediction and "what-if" simulation with a visual curve that changes when inputs change

2) Submission deliverables (what you can submit)

- Full documentation: `docs/assessment5/ASSESSMENT_5_DOCUMENTATION.md`
- Test report (Markdown + PDF): `docs/assessment5/TEST_REPORT.md`, `docs/assessment5/TEST_REPORT.pdf`
- Validation report (Markdown + PDF): `docs/assessment5/VALIDATION_REPORT.md`, `docs/assessment5/VALIDATION_REPORT.pdf`
- Poster outline: `docs/assessment5/POSTER_OUTLINE_A1.md`
- Demo checklist + script: `docs/assessment5/DEMO_CHECKLIST.md`, `docs/assessment5/DEMO_VIDEO_SCRIPT.md`

3) Repository structure

- `data/raw/` — NHANES XPT files (source)
- `data/processed/` — cleaned patient dataset used for modeling
- `src/healthcare_digital_twin/` — reusable Python package (data, modeling, inference, API)
- `scripts/` — CLI entrypoints (`build_dataset.py`, `train.py`, `predict.py`)
- `artifacts/` — trained pipeline(s) + metadata JSON (reproducibility evidence)
- `reports/figures/` — confusion matrix figures saved at training time
- `frontend/` — React UI (Vite) that calls the API

4) Dataset and label

Dataset: NHANES 2017–2018.

Integration approach: multiple NHANES tables are merged by participant identifier `SEQN` to form one patient-level record.

Processed dataset: `data/processed/patient_state_clean.csv`.

Target label: `Metabolic_Risk` created from HbA1c thresholding (as defined in your research work). HbA1c is used only to create the label and is excluded from input features.

5) Feature set (Digital Twin inputs)

The deployed system uses 9 features (also recorded in artifact metadata):

- `Age`
- `Gender` (0=Male, 1=Female)
- `BMI`
- `Systolic_BP`
- `Diastolic_BP`
- `Glucose`
- `Insulin`
- `Total_Cholesterol`
- `HDL_Cholesterol`

6) Modeling approach

6.1 Pipeline

Training and inference use one scikit-learn Pipeline:

1. Median imputation
2. Standard scaling
3. Logistic Regression classifier

This prevents train/serve mismatch and reduces leakage risk.

6.2 Train/test + tuning

- Stratified train/test split (`test_size=0.2` , `random_state=42`)
- Stratified 5-fold CV for tuning
- Primary optimization metric: F1

6.3 Regularization modes (L2 vs L1)

Two artifacts can be produced and served:

- **L2 (default "best" model):** stable baseline performance
- **L1 (sparse model):** encourages sparsity and supports more "explainable" behavior in coefficient-based driver reporting

7) Training + artifacts (CLI)

7.1 Build processed dataset (optional)

```
".venv/bin/python" scripts/build_dataset.py
```

7.2 Train + export L2 artifact

```
".venv/bin/python" scripts/train.py --model logreg --tune --threshold 0.4 --
artifact-name logreg_pipeline
```

7.3 Train + export L1 artifact

```
".venv/bin/python" scripts/train.py --model logreg --tune --penalty l1 --threshold
0.4 --artifact-name logreg_pipeline_l1
```

7.4 Exported outputs

Artifacts (examples):

- `artifacts/logreg_pipeline.joblib`
- `artifacts/logreg_pipeline.metadata.json`
- `artifacts/logreg_pipeline_default.joblib`
- `artifacts/logreg_pipeline_default.metadata.json`
- `artifacts/logreg_pipeline_l1.joblib`
- `artifacts/logreg_pipeline_l1.metadata.json`

Figures (examples):

- `reports/figures/logreg_pipeline.confusion_matrix.png`
- `reports/figures/logreg_pipeline_l1.confusion_matrix.png` (if produced)

8) Evaluation results (from saved artifacts)

Metrics are stored in the metadata JSON for reproducibility.

- Default decision rule (no manual threshold): F1 ≈ 0.6844 (`logreg_pipeline_default.metadata.json`)
- Screening threshold (threshold=0.4): F1 ≈ 0.7090 (`logreg_pipeline.metadata.json`)

Note: The system focuses on the clinically meaningful, user-actionable views (probability, thresholding, confusion matrix, what-if changes). ROC-style plots are intentionally not used in the UI.

9) Backend API (FastAPI)

Start the backend:

```
".venv/bin/python" -m uvicorn healthcare_digital_twin.api:app --reload --port 8000
```

9.1 Endpoints

- `GET /health` → service readiness
- `POST /predict` → risk probability + label
- `GET /model-info` → available models, artifact metadata, dataset overview (dashboard)
- `POST /analyze` → baseline + scenarios with delta probability and top drivers (what-if)
- `GET /figures/{name}` → serves PNG figures from `reports/figures/` (hardened)

9.2 `/predict` contract

Request body: a `PredictionRequest` with the 9 features.

Optional query parameters:

- `model=l2|l1` (select artifact alias)

- `artifact_name=<name>` (direct override)
- `threshold=<float>` (override decision threshold)

Example:

```
curl -s "http://127.0.0.1:8000/predict?model=l2" \
    -H "Content-Type: application/json" \
    -d
'{"Age":45,"Gender":1,"BMI":28.2,"Systolic_BP":130,"Diastolic_BP":82,"Glucose":110,"Ins
```

9.3 `/analyze` contract (what-if)

`/analyze` accepts a baseline patient and a list of scenario overrides. It returns:

- baseline risk probability + drivers
- scenario risk probability + delta vs baseline + drivers

This endpoint powers the UI's probability-vs-Δ curve.

9.4 CORS / frontend compatibility

Local dev CORS is configured to allow common Vite origins (e.g., ports `517x`).

10) Frontend (React dashboard)

Start the UI:

```
cd frontend
npm install
npm run dev
```

Open: `http://127.0.0.1:5173` .

Optional backend URL override:

- Copy `frontend/.env.example` → `frontend/.env.local`
- Set `VITE_API_BASE_URL=http://127.0.0.1:8000`

10.1 Implemented UI features

- Input form with validation (min/max checks aligned to API schemas)
- Model selector: L2 (default) vs L1 (sparse)
- Prediction result with probability + threshold + risk label
- What-If simulator:
  - user changes one feature by Δ
  - UI calls `/analyze` and displays the probability change
  - UI renders a **dynamic curve** (probability vs Δ) that visibly changes with different inputs
  - UI shows top drivers for the selected scenario
- Model dashboard:
  - dataset overview (rows, positive rate, female rate)
  - model metrics (from metadata)
  - confusion matrix (test set) image served via `/figures/{name}`

11) Testing and quality gates

11.1 Backend automated tests

```
.venv/bin/python -m pytest -q
```

Current executed result: **9 passed**.

11.2 Frontend gates

```
cd frontend
npm run lint
npm run build
```

Both pass in the current environment.

11.3 Reports

- Test report: `docs/assessment5/TEST_REPORT.md` and `docs/assessment5/TEST_REPORT.pdf`
- Validation report: `docs/assessment5/VALIDATION_REPORT.md` and `docs/assessment5/VALIDATION_REPORT.pdf`

12) Demo flow (what to present)

1. Start backend → show `GET /health`
2. Open UI → show API connected badge
3. Enter patient inputs → run prediction → show probability + label
4. Switch model (L2 ↔ L1) → show metrics/dashboard update
5. Run what-if (e.g., increase Glucose) → show Δ probability and the curve update
6. Show confusion matrix figure as “test set evaluation” (does not change per single patient)

13) Limitations (honest academic notes)

- NHANES is cross-sectional; this is a statistical risk estimator for demo/academic use.
- Not validated for clinical decision-making.
- External generalization requires evaluation on local populations.

14) Troubleshooting

- If `uvicorn` fails to start:
  - confirm dependencies: `".venv/bin/python" -m pip install -r requirements.txt`
  - confirm editable install: `".venv/bin/python" -m pip install -e .`
  - confirm port 8000 is free, or use `--port 8001`
- If the UI shows API disconnected:
  - confirm backend is running
  - confirm `VITE_API_BASE_URL` matches the backend
  - CORS is preconfigured for local dev origins, but custom origins must be allowed

15) Appendix: PDF export commands

To export documentation/reports to PDF (used for submission):

```
npx --yes md-to-pdf docs/assessment5/ASSESSMENT_5_DOCUMENTATION.md
npx --yes md-to-pdf docs/assessment5/TEST_REPORT.md
npx --yes md-to-pdf docs/assessment5/VALIDATION_REPORT.md
```